MODULE 10

Agents as Your Team: Foundations

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 2 / Second**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Explain what Claude Code subagents are and how they differ from standard Claude Code prompts and headless mode
2. Configure subagents via YAML frontmatter in .claude/agents/ Markdown files with custom system prompts and tool permissions
3. Design a team of subagents with specialized roles: planner, reviewer, implementer, and tester
4. Apply guardrails including tool allow-lists, read-only mode, model selection, and sandbox isolation
5. Configure loop exit criteria and error recovery patterns for autonomous agent workflows
6. Use background tasks (Ctrl+B) for long-running subagent processes without blocking the main session

SESSION TIMING BREAKDOWN

0:00 – 0:08	**Section 1: What Are Subagents?**
0:08 – 0:18	**Section 2: Configuring Subagents with YAML**
0:18 – 0:28	**Section 3: Guardrails, Permissions & Safety**
0:28 – 0:37	**Section 4: Loop Exit Criteria & Error Recovery**
0:37 – 0:43	**Section 5: Background Tasks & Practical Patterns**
0:43 – 0:45	**Wrap-Up: Quick Reference & Transition to Module 11**

0:00 – 0:08	SECTION 1: What Are Subagents?

1. What Are Subagents?

1.1 From Tool to Team

So far, you've used Claude Code as a single assistant handling everything — scaffolding, coding, testing, debugging, and CI/CD. Subagents change the model: instead of one generalist, you build a team of specialists, each with their own expertise, context window, tool permissions, and system prompt.

Standard Claude Code	Subagents
One AI, one context window	Multiple specialists, each with own context
Handles everything in one conversation	Each agent focuses on one type of task
Context gets crowded on complex tasks	Isolation prevents cross-contamination
All tools available to every prompt	Each agent has only the tools it needs
You manage everything	Agents delegate to each other automatically

1.2 The Agent Team Model

Think of subagents as roles on a development team. Each has a specific job:

Role	What It Does	Tool Access
Planner	Reads codebase, creates implementation plan, identifies affected files	Read, Grep, Glob (read-only)
Reviewer	Analyzes code changes for bugs, security, and style violations	Read, Grep, Glob, Bash
Implementer	Writes code following the plan, creates/modifies files	Read, Write, Edit, Bash
Tester	Generates tests, runs test suite, reports coverage	Read, Write, Bash, Glob
Documenter	Generates docstrings, README updates, API documentation	Read, Write, Edit, Glob

☆ **KEY POINT:** Each subagent runs in its own context window. This is the key advantage: the planner can explore your entire codebase without polluting the implementer's context. When the planner finishes, it returns a concise summary, and the implementer works from that summary — not the full exploration history.

1.3 How Subagents Are Invoked

```
# Automatic delegation (Claude recognizes the task):
> Review the changes I just made to validators.py
```

```
# Claude auto-delegates to code-reviewer subagent

# Explicit invocation:
> Use the code-reviewer subagent to analyze my latest commits

# Via /agents command:
> /agents  (opens the agent management UI)
```

0:08 – 0:18	SECTION 2: Configuring Subagents with YAML

2. Configuring Subagents with YAML

2.1 Anatomy of a Subagent File

Subagents are Markdown files with YAML frontmatter. They live in .claude/agents/ (project-level) or ~/.claude/agents/ (user-level). Project agents take priority over user agents when names conflict.

```
# .claude/agents/sensor-reviewer.md
---
name: sensor-reviewer
description: Reviews sensor toolkit code for quality,
  TI coding standards, and data validation correctness.
  Use PROACTIVELY after code changes.
tools: Read, Grep, Glob, Bash
model: inherit
---

You are a senior code reviewer specializing in sensor
data processing at Texas Instruments.

When invoked:
1. Run `git diff` to identify modified files
2. Check against TI coding standards (Google Python Style)
3. Verify all validation ranges match sensor specifications
4. Ensure type hints and docstrings are present
5. Check test coverage for modified functions

Provide feedback organized by priority:
- **Critical** (must fix before merge)
- **Warning** (should fix soon)
- **Suggestion** (consider improving)

Return a JSON summary with: files_reviewed, issues_found,
coverage_status.
```

2.2 YAML Frontmatter Fields

Field	Description
name (required)	Unique identifier. Use kebab-case: sensor-reviewer, test-generator
description (required)	When to use this agent. Claude reads this to decide auto-delegation. Be action-oriented.
tools (optional)	Comma-separated tool list. Omit to inherit all main thread tools. Restrict for safety.
model (optional)	Which Claude model to use: sonnet, opus, haiku, or inherit. Use lighter models for simple tasks.
--- body ---	The system prompt. This is ALL the subagent sees — plus basic env details like working directory.

2.3 Creating Subagents for the Sensor Toolkit

```
# Create the agents directory:
mkdir -p .claude/agents

# Agent 1: Planner (read-only)
cat > .claude/agents/sensor-planner.md << 'EOF'
---
name: sensor-planner
description: Plans implementation for sensor toolkit features.
  Use before writing any new code.
tools: Read, Grep, Glob
model: inherit
---
You are a senior Python architect. When given a feature request:
1. Analyze the existing codebase structure
2. Identify which files need changes
3. Create a step-by-step implementation plan
4. List edge cases and potential risks
Output a structured plan in Markdown.
EOF

# Agent 2: Test Generator
cat > .claude/agents/sensor-tester.md << 'EOF'
---
name: sensor-tester
description: Generates comprehensive tests for sensor toolkit.
  Use after code changes to ensure quality.
tools: Read, Write, Bash, Glob, Grep
model: inherit
---
You are a testing specialist. When invoked:
1. Analyze the target module for testable functions
2. Generate pytest tests using BICEP framework
3. Use fixture factories from conftest.py
4. Run tests and report coverage
5. Add tests for any uncovered lines
Target: 80%+ branch coverage.
EOF
```

Track A (IT Engineers)	Track B (Process Engineers)
Create a ci-checker subagent that validates GitHub Actions YAML and Dockerfile syntax	Create a data-validator subagent that checks CSV schemas and sensor ID formats
Create a security-scanner subagent with read-only tools that identifies API key leaks and SQL injection	Create a report-generator subagent that summarizes sensor data quality metrics
Create a pr-writer subagent that generates PR descriptions from git diff	Create a pipeline-monitor subagent that checks data pipeline output directories

0:18 – 0:28	SECTION 3: Guardrails, Permissions & Safety

3. Guardrails, Permissions & Safety

3.1 The Principle of Least Privilege

Every subagent should have only the tools it needs. A reviewer should never be able to Write files. A planner should never execute Bash. This prevents accidental damage and enforces role separation.

Agent Role	Allowed Tools	Blocked (Implicit)
Planner	Read, Grep, Glob	Write, Edit, Bash, WebFetch
Reviewer	Read, Grep, Glob, Bash (read-only commands)	Write, Edit
Implementer	Read, Write, Edit, Bash, Glob, Grep	(full access, but scoped by system prompt)
Tester	Read, Write, Bash, Glob, Grep	Edit (tests should be new files, not edits)
Documenter	Read, Write, Edit, Glob, Grep	Bash (no need to run code)

3.2 Model Selection for Cost Control

```
# Use lighter models for simple tasks:
# sensor-reviewer.md frontmatter:
model: sonnet       # Good enough for code review

# sensor-planner.md frontmatter:
model: opus         # Needs deep reasoning for architecture

# sensor-tester.md frontmatter:
model: inherit      # Use whatever the main session uses

# Environment variable override for all subagents:
export CLAUDE_CODE_SUBAGENT_MODEL="claude-sonnet-4-5-20250929"
# Main session uses Opus, subagents use Sonnet (cost savings)
```

☆ **KEY POINT:** A common pattern: run your main session on Opus for complex reasoning while subagents handle focused tasks on Sonnet. This cuts API costs significantly without sacrificing quality on well-scoped agent work. Monitor your usage with the cost tracking in JSON output.

3.3 Sandbox Isolation

For high-risk tasks, subagents can run in sandboxed environments:

```
# Restrict Bash commands with allowedTools patterns:
# In .claude/settings.json:
{
  "permissions": {
    "allowedTools": [
```

```
      "Read",
      "Grep",
      "Bash(pytest *)",
      "Bash(ruff *)",
      "Bash(black *)"
    ],
    "disallowedTools": [
      "Bash(rm *)",
      "Bash(git push *)",
      "Bash(pip install *)"
    ]
  }
}
```

⚠ **WARNING:** Never give subagents unrestricted Bash access in production environments. A subagent with full Bash can delete files, install packages, or push to remote repos. Always use allowedTools patterns to restrict Bash to specific commands (pytest, ruff, black, git diff).

0:28 – 0:37	SECTION 4: Loop Exit Criteria & Error Recovery

4. Loop Exit Criteria & Error Recovery

4.1 Agent Loops

When you chain subagents into a workflow (planner → implementer → tester → reviewer), each agent runs and returns results to the orchestrator (your main Claude Code session). The orchestrator decides what to do next. But what if an agent gets stuck, produces bad output, or runs too long?

Problem	Solution
Agent runs too long (infinite exploration)	Set timeout: timeout 60 claude -p "..." or use token budget in CLAUDE.md
Agent produces incorrect output	Add validation step: check output format, run tests, verify assertions
Agent fails with error	Catch errors in wrapper script, retry with modified prompt, or escalate to human
Agent enters retry loop	Set max_retries in wrapper script, exit after N attempts
Circular delegation	Subagents cannot spawn other subagents (built-in protection)

4.2 Error Recovery Patterns

```
#!/bin/bash
# Wrapper script with retry and error recovery:
MAX_RETRIES=3
TIMEOUT=120

for attempt in $(seq 1 $MAX_RETRIES); do
  echo "Attempt $attempt of $MAX_RETRIES"

  # Run the subagent with timeout
  timeout $TIMEOUT claude -p \
    "Use the sensor-tester subagent to generate tests for \
     @src/sensor_toolkit/validators.py" \
    --output-format json > result.json 2>&1

  EXIT_CODE=$?

  # Check for success
  if [ $EXIT_CODE -eq 0 ] && \
     ! jq -e '.is_error' result.json > /dev/null 2>&1; then
    echo "Success on attempt $attempt"
    # Validate: run the generated tests
    pytest tests/ -v --tb=short
    if [ $? -eq 0 ]; then
      echo "All tests pass. Done."
      exit 0
    fi
  fi

  echo "Attempt $attempt failed. Retrying..."
  sleep 2
```

```
done

echo "All $MAX_RETRIES attempts failed. Human review needed."
exit 1
```

☆ **KEY POINT:** The key pattern: run agent → validate output → test result → retry or escalate. Never let an agent loop indefinitely. Always have a maximum retry count and a human escalation path. In production, log every attempt for debugging.

4.3 Chaining Agents with Hooks

```
# Use hooks to chain agents automatically:
# In .claude/settings.json:
{
  "hooks": {
    "SubagentStop": [
      {
        "matcher": "sensor-planner",
        "hooks": [{
          "type": "command",
          "command": "echo 'Planner done. Next: Use the sensor-tester subagent.'"
        }]
      }
    ]
  }
}

# The hook prints the next step to Claude's transcript,
# prompting it to continue the pipeline automatically.
```

0:37 – 0:43	SECTION 5: Background Tasks & Practical Patterns

5. Background Tasks & Practical Patterns

5.1 Background Tasks

Some subagent tasks take a long time: codebase exploration, security audits, documentation generation. You don't want to wait. Background tasks let agents work while you continue coding:

```
# Press Ctrl+B during any subagent execution to background it

# Or configure automatic backgrounding in CLAUDE.md:
## Background Execution Rules
Run in background automatically:
- Codebase exploration and analysis
- Security audits and performance profiling
- Documentation generation
- Any task where results aren't needed immediately

# Check background task status:
> /tasks

# Resume a backgrounded task:
> /tasks  (select the task to bring to foreground)
```

5.2 Practical TI Subagent Patterns

Pattern	Subagents	Workflow
Feature Development	planner → implementer → tester → reviewer	Plan first, implement, test, review. Gate at each step.
Bug Fix	debugger → implementer → tester	Paste traceback → diagnose → fix → verify with tests.
Code Review	reviewer (auto on every PR)	Runs automatically via CI hook. Posts findings as PR comment.
Release	changelog-writer → reviewer	Generate release notes → human reviews before publishing.
Data Quality	data-validator (auto on push to data/)	Checks CSV schemas, sensor ID formats, value ranges.

☆ **KEY POINT:** Start simple: create one or two subagents for your most repeated tasks (code review, test generation). Once those are working well, add more agents and chain them. Don't try to build a full pipeline on day one — iterate.

0:43 – 0:45	WRAP-UP: Quick Reference & Transition to Module 11

6. Quick Reference & Transition

MODULE 10 RECAP

✅ Subagents are specialized AI assistants with their own context window, system prompt, tools, and model selection.

✅ Defined as Markdown files with YAML frontmatter in .claude/agents/ (project) or ~/.claude/agents/ (user).

✅ Key YAML fields: name, description (for auto-delegation), tools (least privilege), model (cost control).

✅ Principle of least privilege: planners get Read-only, implementers get Write, reviewers get Read + Bash.

✅ Model selection: Opus for complex reasoning, Sonnet for focused tasks, Haiku for simple checks.

✅ Error recovery: timeout + retry + validation + human escalation. Never let agents loop indefinitely.

✅ Chain agents with SubagentStop hooks. Background long-running tasks with Ctrl+B.

✅ Start simple: one or two agents for repeated tasks. Iterate from there.

QUICK REFERENCE CARD

Task	Command / Config	Notes
Create subagent	mkdir -p .claude/agents && vi sensor-reviewer.md	YAML frontmatter + Markdown body
List subagents	/agents	Shows all loaded agents
Invoke explicitly	> Use the sensor-reviewer subagent to review validators.py	Name must match
Auto-delegation	Claude reads description field to match tasks	Write clear descriptions
Restrict tools	tools: Read, Grep, Glob	Omit = inherit all
Set model	model: sonnet / opus / haiku / inherit	Cost vs. quality trade-off
Background task	Ctrl+B during execution	Check with /tasks
Chain via hooks	SubagentStop hook in .claude/settings.json	Prints next step to transcript
Error recovery	timeout 60 claude -p + retry loop in bash	Always set max retries

TRANSITION TO MODULE 11

You now understand the fundamentals of subagents. Module 11 (Agents for Software Development: Applied) puts them to work: repository observation, file localization with intelligent grep/glob, the Plan-Execute-Review cycle, safety metrics for agent-generated code, and MCP server integration for extending Claude Code's capabilities beyond your local filesystem.